

Protocol Design and Empirical Characterisation of a Data Channel over the Find My network

Nguyen Thai Binh

CP2107

August 10, 2026

Contents

1. Introduction	3
1.1. Motivation	3
1.2. Prior Works	3
1.3. Contributions	3
2. Background	5
2.1. Find My Protocol	5
2.2. NIST P-224 Elliptic Curve	5
2.3. BLE Legacy Advertising	6
3. The sendmy Data Protocol	8
3.1. Link Layer: Emitting a Valid Advertisement	8
3.2. Carrier Layer: Encoding an Arbitrary Byte	8
3.3. Reference Applications	9
4. Characterising the Channel: The espbench Test Bench	10
4.1. Questions	10
4.2. Architecture	10
4.3. Experimental Controls	11
5. Results	13
5.1. Erasure Without Corruption	13
5.2. Repetition Versus Power	13
5.3. Rotation and Broadcast-Rate Invariance	14
5.4. The 300 ms Deliverability Nulls	14
5.5. Relay-Bound Latency	17
5.6. The Rate–Reliability Trade-Off	19
6. Discussion and Interpretation	20
7. Limitations and Future Work	21
8. Ethics and Responsible Use	22
8.1. Network Load	22
8.2. Privacy of Finder-Device Owners	22
8.3. Dual-Use Considerations	22
8.4. Use of Artificial Intelligence (AI)	22
9. Conclusion	24
Bibliography	25
Index of Figures	26
Index of Tables	26

1. Introduction

1.1. Motivation

Apple currently operates one of the largest crowd-sourced Bluetooth Low Energy (BLE) location-relay networks, powered by hundreds of millions of Find My-enabled devices. In parallel, there exists a large class of low-power embedded devices carrying only a 2.4 GHz radio for BLE communication. Such a device can advertise to nearby listeners but cannot, on its own, reach the wider internet.

The network exists to allow owners of compatible Apple devices to locate lost or stolen objects without Internet connectivity; however, its mechanics — finders relaying valid BLE advertisements to the backend — are exactly those of a one-way message-passing service that allows a BLE-only device to piggyback off the network and send data to the Internet without any infrastructure of the sender’s own.

1.2. Prior Works

That this is possible at all has been shown before. Prior proof-of-concept work demonstrated that arbitrary bytes can be smuggled through such a network by encoding them into the advertised key material and querying the backend for their reappearance [1], and open reimplementations of the offline-finding cryptography made the mechanism inspectable and reproducible [2]. What that work established is feasibility. This report expands on that work and analyses the **behaviour** instead: how *reliable* the channel is, how *quickly* a message arrives, which transmitter parameters actually matter, and *how much data per second* can be pushed through. Those questions must be answered quantitatively before building a reliable, efficient opportunistic network.

1.3. Contributions

My contributions are as follows.

1. **sendmy** — a two-layer protocol that, unlike previous works, decouples encoding from transmission and keeps payloads confidential end to end under a 32-byte pre-shared key (Section 3);
2. **espbench** — an unattended test bench and host harness whose design takes into account the statistical hazards of measuring a shared, drifting, crowd-sourced medium as well as the unique challenges of an unreliable one-way network;
3. Empirical characterisation of the channel, which shows that it is a near-lossless erasure channel at full power, numbers of broadcasts per key dominate deliverability, latency is *mostly* finder-side and heavily right-tailed, and there exist periodic deliverability nulls at advertising intervals near multiples of 300 ms, with a clean explanation in how the interval lines up against the scanner’s listening cycle (Section 5 and Section 6); and finally,
4. Practical transmit rules for network building and a methodological warning about adaptive-timeout measurement of heavy-tailed processes (Section 6 and Section 7).

Table 1 collects the notation and terminology I use throughout.

SYMBOL	MEANING
A	Advertising interval: nominal time between successive advertising events.
T	Key rotation period: time a single advertised key is broadcast before rotation.
$B = T/A$	Broadcasts per key: number of advertising events emitted per key.
Transmit power	Radio output level of the transmitter, quoted in dBm; low (-24 dBm) and high ($+9$ dBm) settings are compared throughout.
Key	28-byte elliptic-curve point (NIST P-224) serialised into an advertisement; the addressable unit a report is filed against.
Deliverability	Fraction of transmitted keys for which a fetchable report later exists.
Propagation latency	Elapsed time from a key's first broadcast to its first fetchable report, covering both discovery and relay upload.
Existence-oracle query	Backend lookup for which candidate key has a report on file.
Erasur channel	Channel whose only failure mode is loss; a delivered key is never corrupted.
Goodput	Delivered payload bytes per second of wall-clock on-air time.
Live poller	Two-tier real-time detection mechanism that tracks progress during a run but is not authoritative.
UART	Universal asynchronous receiver-transmitter: the wired host-to-board link over which the test bench configures the transmitter and collects its log.

Table 1: Notation and terminology used throughout. A transmitter is characterised by the triple (A, T, B) together with its transmit power.

2. Background

2.1. Find My Protocol

Apple’s **Find My** network - the part relevant here is sometimes called **Offline Finding (OF)** - lets a lost device be located even when it has no Internet connection of its own. The mechanism works roughly as follows:

1. A lost device continuously broadcasts a BLE advertisement containing a rotating public key.
2. Any nearby Apple device (“finder”) that hears the advertisement encrypts its own current location to that public key and uploads the encrypted report to Apple’s servers.
3. The owner of the lost device, who knows the corresponding private key, later downloads and decrypts the reports to recover the device’s location history.

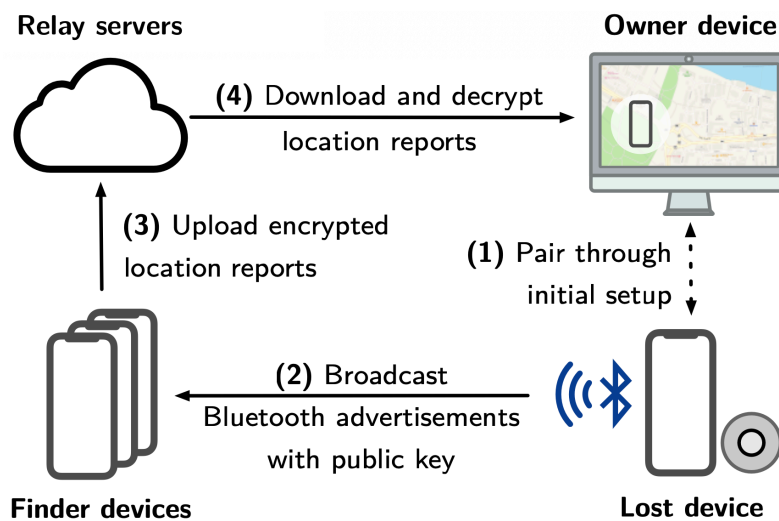


Figure 1: Simplified OF workflow [3]

Two properties of this design make it attractive to repurpose. First, the finder relays beacons *opportunistically and anonymously* - it does not authenticate the lost device. Second, because the location is encrypted to a key the finder cannot read, Apple’s network is content-agnostic about what it carries. The advertised “public key” is, from the network’s point of view, just an opaque blob that happens to be a valid point on a particular elliptic curve.

2.2. NIST P-224 Elliptic Curve

Due to the limited payload size, Apple opts to use the NIST P-224 elliptic curve as its cryptographic primitive, which has a compressed public key size of 29 bytes¹. In theory, this should fit comfortably within the 33-byte payload limit of a legacy BLE advertisement (see Section 2.3). However, the Find My protocol includes additional metadata, such as battery level and device conditions. As a result, the protocol strips the most significant byte from the compressed public key before packing it into the advertisement, resulting in a payload size of 28 bytes.

¹According to NIST standard, the compressed public key of P-224 ECC consists of the 28-byte x -coordinate of the public point prepended with one additional byte for the sign y -coordinate (`0x02` for positive, `0x03` for negative).

2.3. BLE Legacy Advertising

The Find My protocol — and therefore the channel built on it — rests entirely on the legacy (non-connectable) advertising mode of BLE, as defined in the core specification [4]. In this mode a device is a **broadcaster**: it periodically transmits an advertising packet that any **observer** in range may receive, with no connection, association, or acknowledgement. Figure 2 shows the structure of a BLE advertising packet.

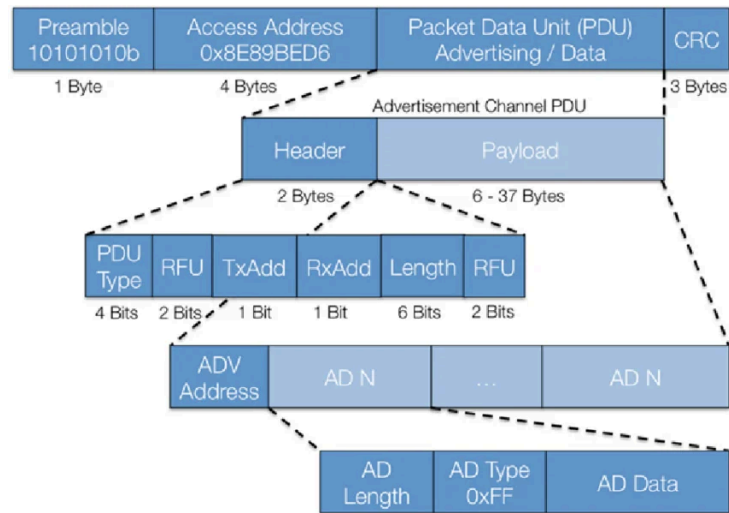


Figure 2: BLE advertising packet structure. ADV Address (2nd last row) and AD Data (last row) fields are what we can freely set.

Since the advertising data (AD) field can contain at most 31 bytes (only 27 bytes are actually usable), together with the 6-byte device address, a BLE advertisement may encode a payload of at most 33 bytes of data². Table 2 shows how Apple splits the 28-byte key into two parts: the first 6 bytes become the BLE device address, and the remaining 22 bytes — together with the displaced top two bits of the first byte — are packed with other metadata into the 31-byte AD field.

NAME	SIZE (B)	DESCRIPTION/VALUE
AD length	1	0x1F (31 bytes)
AD type	1	0xFF (Manufacturer-specific data)
Company ID	2	0x004C (Apple)
OF type	1	0x12
OF length	1	0x19 (25 bytes)
Status	1	0x00 (Metadata)
Key[6..27]	22	Last 22 bytes of the 28-byte key
Key[0] >> 6	1	Top 2 bits of Key[0]
Hint	1	0x00 (Unknown purpose)

²The two most significant bits of the device address must be set to 0b11 to indicate it is a static random address, as mandated by the BLE specification. Thus, as shown later, these two bits take up one additional byte in the Find My protocol.

Table 2: Find My BLE advertising payload format. The first 6 bytes of the key are used as the BLE device address.

3. The `sendmy` Data Protocol

`sendmy` is organised into two layers. The **link layer** places an arbitrary 28-byte key onto the air as a structurally valid Find My advertisement. The **carrier layer** sits above it and decides *which* key to advertise, so that a one-byte payload can be recovered by a receiver who never sees the transmitter directly. The two reference applications sit above both.

3.1. Link Layer: Emitting a Valid Advertisement

The relay network files reports against the advertised public key, so the addressable unit of the channel is a single 28-byte key — the size of a public key on the curve the Find My format uses. The link layer has one job: to serialise a given 28-byte key into a well-formed advertising event, and to rotate keys safely.

The 28 bytes do not fit in any single AD field, so the wire format scatters them across the two settable regions of Table 2: the six bytes of the BLE static-random device address, and a 22-byte region (plus two bits) of a manufacturer-specific AD record. The remaining bits of the address carry the fixed pattern that marks it as static-random, and the AD record’s framing bytes reproduce the structure finder devices expect. To any scanner, the result is an ordinary Find My advertisement.

The link layer exposes a small interface — `sm_ll_init`, `sm_ll_set_key`, `sm_ll_set_tx_power`, and `sm_ll_set_adv_interval` — and guarantees that `sm_ll_set_key` is thread-safe against the advertising task, so the layer above can rotate the advertised key at any moment without racing the radio. Key rotation is therefore a first-class operation, which the carrier layer relies on to step through a sequence of keys.

3.2. Carrier Layer: Encoding an Arbitrary Byte

The carrier layer turns a one-byte payload into a 28-byte key, in such a way that a receiver holding the protocol, the message-identifier schedule, and a shared secret can recover the byte. The **message identifier** (`mid`) is a 32-bit counter that names each one-byte message on the channel. No handshake synchronises it: the sender advances it monotonically, one identifier per message, and the receiver, following the same schedule, resolves one existence-oracle query per `mid` to recover the bytes in order.

The shared secret is a **32-byte pre-shared key**, which I call the `uid`. It is generated once and installed on both the sender and the receiver, and it is the only secret the two share. Encoding a payload then proceeds in three steps.

1. The `uid` is used directly as the pseudorandom key of an HKDF-Expand step [5], itself built on HMAC [6]. The Extract stage is skipped, since a random 32-byte `uid` is already a uniform key.
2. The expansion runs over the info string `"smv1" || mid_be32 || payload || attempt`, where `mid_be32` is the big-endian message identifier, `payload` is the byte being sent, and `attempt` is a rejection-sampling counter.
3. The 28-byte output block is read as a big-endian integer and taken as a scalar d . The transmitted carrier is the big-endian x -coordinate of the point $d \cdot G$ on NIST P-224 / secp224r1 [7], where G is the standard base point.

A raw 28-byte block is not guaranteed to be a valid in-range scalar — it may equal or exceed the group order — so when it is out of range the `attempt` counter is incremented and the expansion repeated until the scalar is valid. Rejection sampling therefore guarantees that the carrier is always a genuine scalar multiple of G , i.e. a point on the curve.

This guarantee is what makes the channel a clean **erasure channel**. Since every carrier the transmitter can emit is a valid point by construction, a receiver’s failure to find a report for a key cannot be an encoding fault; it can only mean that no finder device relayed that key. A miss is always network loss. The standardised alternative would be a hash-to-curve map [8], which sends an arbitrary string to a curve point in constant time and without rejection. I prefer rejection here for three reasons: the expected number of retries is negligible (the group order is very close to 2^{224} , so a rejection occurs with probability about 2^{-112}); the construction reuses the HKDF primitive already in the design; and that same rarity means the rejection loop leaks nothing usable through timing.

Recovery is an **existence oracle** over the relay backend. Because the payload is a single byte, a given mid has only 256 possible carriers. The receiver derives all 256 locally — running the same HKDF-to-point construction under the shared uid for each payload value — and asks the backend which of the 256 keys has a report on file. The one that does reveals the transmitted byte; the backend itself does the heavy lifting of matching a relayed key to a stored report.

The uid is also what makes the channel private end to end. To anyone without it — a BLE eavesdropper, a protocol-aware observer, or the backend operator — the 256 candidates are not computable, because without the HMAC key the candidate space is the whole curve rather than 256 points. The carrier is then indistinguishable from a random public key, and the private scalar d needed to decrypt the finder-side location reports cannot be derived. Both the payload and the location reports are therefore readable only by holders of the uid.

3.3. Reference Applications

Two reference applications in the `sendmy` repository^o exercise the protocol. `esptag` (examples/esptag^o) is a rotating locator-tag beacon that simply advertises a sequence of keys, standing in for a generic Find My device. `espsend` (examples/espsend^o) is a temperature sender: it transmits one signed byte of temperature per message identifier, advancing the identifier on each new reading, and a receiver reconstructs the temperature time series by resolving one oracle query per identifier. Both are built on ESP-IDF [9] and the NimBLE host stack, and run on a Seeed XIAO ESP32-S3 — a compact module representative of the BLE-only devices the channel is meant to serve.

4. Characterising the Channel: The `espbench` Test Bench

With feasibility already established by prior work, the contribution of this report is a measured characterisation of the channel. The medium is a live, shared, drifting, crowd-sourced network that cannot be instrumented from the inside, so the measurement apparatus is itself a substantial part of the work. This chapter sets out the questions, the architecture of the test bench, and the experimental controls that shaped its design.

4.1. Questions

The test bench is built to answer four questions.

- **Deliverability.** What fraction of transmitted keys are ultimately retrievable, and under what conditions?
- **Latency.** How long after a broadcast does a report become fetchable, and is the delay in discovery (a finder device hearing the advertisement) or in propagation (the report reaching the backend)?
- **Which parameter matters.** Of the advertising interval A , the key rotation period T , the broadcasts per key B , and the transmit power, which actually move deliverability, and which are inert?
- **Goodput.** How many payload bytes per second of on-air time can the channel carry, and at what reliability?

4.2. Architecture

The firmware is flashed once and thereafter reconfigured over UART: the host harness sets A , T , the transmit power, and the key schedule at runtime, with no reflashing, which is what makes an unattended overnight sweep possible. The harness drives a **parameter matrix** — a set of experimental **cells** — and runs it for hours without supervision.

The **cell** is the experimental unit of everything that follows. Each cell is one combination of transmitter parameters, sent as roughly twenty consecutive keys within a single contiguous time window, under its own freshly minted 32-byte uid — the pre-shared carrier-layer secret of Section 3.2 — and its own disjoint range of message identifiers. The last two give structural independence across cells on a shared relay: because each cell derives its keys under a different uid over a disjoint mid range, no two cells can ever contend for or collide on the same backend key.

For every key, the harness records an append-only time series capturing (i) the transmission event; (ii) each live detection, including the decrypted location report — latitude, longitude, accuracy, confidence, and status — whenever one is seen; (iii) the offline deliverability sweep; and (iv) the ambient finder density, sampled over time. Two mechanisms observe delivery. A two-tier **live poller** watches for reports as the run proceeds, giving a real-time view of progress. Then — and this is the authoritative measurement — an **offline resweep** re-fetches every key after the fact, with no queue cap and unbounded patience. It is the resweep, never the live poller, that defines deliverability ground truth.

Ambient finder density was logged throughout the run, and Figure 3 shows it. Density follows a day–night cycle: it falls to roughly 9–13 finder devices in the afternoon and evening trough

and rises to about 22 in the early-morning peak. This variation modulates the channel — it is exactly the drift the round-robin interleaving (below) is designed to spread evenly — but, as Section 5 shows, it does not explain the deliverability effects, which persist across the whole cycle.

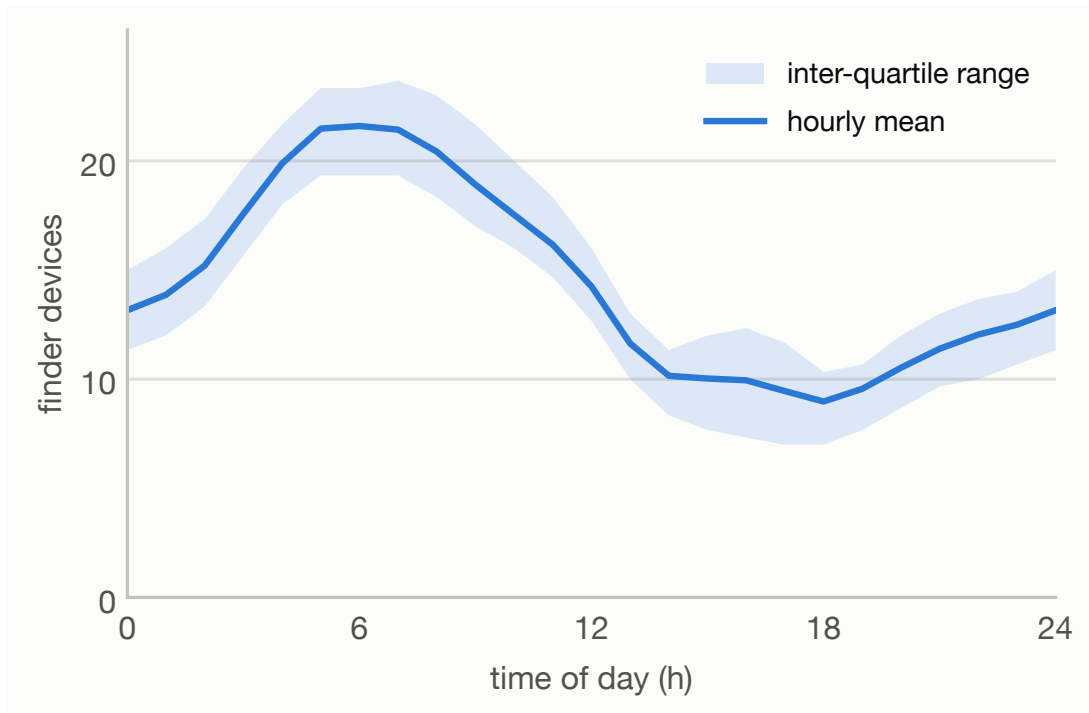


Figure 3: Ambient finder density over the course of a run, sampled continuously as a covariate. Density follows a day-night cycle, with an afternoon and evening trough and an early-morning peak.

4.3. Experimental Controls

Several controls defend the measurements against the peculiarities of an overnight, crowd-sourced medium.

- **Round-robin interleaving.** Conditions are interleaved rather than run in blocks, so that the slow day-night drift in finder density (Section 5) is spread evenly across all cells, instead of confounding whichever condition happened to run at 3 a.m.
- **Anchor cells.** A fixed reference condition is repeated throughout, making up roughly 7–10% of each run, as an instrument check: if the anchors misbehave, the night is suspect.
- **Cell-clustered testing.** The keys within a cell share a uid, a time window, and hence a density history, so they are not independent samples. Significance is therefore assessed by permutation tests that resample at the level of the cell, not the key.
- **Continuous density logging.** Finder density is logged throughout so that a run of misses can be attributed correctly, distinguishing a genuinely empty network from one that was present but dropped the key.

One rule follows from the anchors and governs how the results should be read. The anchor condition is chosen for stability rather than sensitivity — deliberately far up the reliability curve, where ambient variation barely moves it — so the anchors can flag a bad night but

cannot calibrate a difference **between** nights. Accordingly, I treat only **within-run** contrasts as effects, and never compare absolute rates across runs.

5. Results

I fix the baseline first, then take deliverability, latency, and goodput in turn. Throughout, absolute rates are quoted within a run and are not compared across runs.

5.1. Erasure Without Corruption

At full transmit power and urban finder density, the channel is lossless for practical purposes: per-run deliverability sits between about 99.5% and 99.94%. More striking than the loss rate is the error behaviour. Across every run and every cell, **no payload corruption was ever observed** — every delivered key decoded to exactly the byte that was sent. This follows directly from the carrier construction of Section 3.2: a key either reappears intact or does not reappear at all. The channel is thus a **pure erasure channel**, whose only failure mode is loss, and the receiver always knows when a loss has occurred, because the oracle query for the expected message identifier returns nothing.

5.2. Repetition Versus Power

The single most effective control on deliverability is the number of broadcasts per key, B . Figure 4 shows deliverability as B rises from 1 to 16 at two transmit powers. Misses decay roughly geometrically in B : at -24 dBm deliverability climbs through 42.8, 66.1, 83.3, 97.2, and 100% as B goes 1, 2, 4, 8, 16, while at $+9$ dBm it starts high at $B = 1$ (86.1%) and saturates quickly (98.3, 96.7, 100, 100%).

This is the shape of independent Bernoulli chances to be heard: each extra broadcast is another opportunity for some finder device to catch the key, so the residual miss probability falls off like a power of a per-broadcast miss rate. The low-power arm bears this out. The per-broadcast miss probability each point implies on its own, $(1 - d)^{1/B}$, reads 0.57, 0.58, 0.64, and 0.64 at $B = 1, 2, 4, 8$ — near enough constant that a single per-broadcast miss probability of about 0.6 reproduces the whole sweep to within some four points. The full-power arm saturates too fast, and is not even ordered between $B = 2$ and $B = 4$, for the same exercise to say much. Transmit power does not change this shape; it sets **where** the curve saturates, by raising the per-broadcast probability that any given event is heard. The practical reading is simple: B is the knob to turn for reliability, and power decides how few broadcasts suffice.

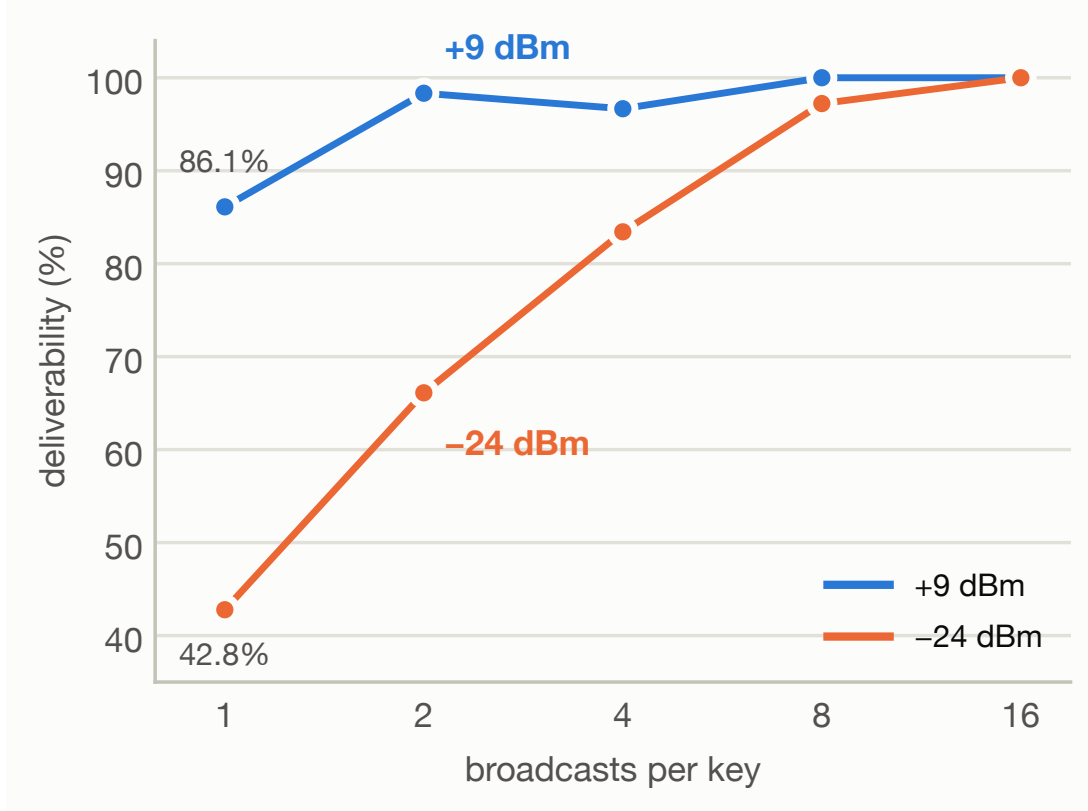


Figure 4: Deliverability against broadcasts per key $B \in \{1, 2, 4, 8, 16\}$ at -24 dBm and $+9$ dBm. Misses decay approximately geometrically in B .

5.3. Rotation and Broadcast-Rate Invariance

At full power the channel sits at its deliverability ceiling, and there the key rotation period T and the raw broadcast rate are inert: varying them produces no resolvable change, because there is no headroom above the ceiling for them to occupy. This is not a claim that T never matters — it matters for latency (Section 5.5), and it would matter for deliverability in a power-starved regime — but at full power it is hidden beneath the ceiling.

5.4. The 300 ms Deliverability Nulls

The most surprising finding concerns the advertising interval A . One might expect deliverability to vary smoothly with A ; instead it shows sharp, regularly spaced deficits, which I call **periodic deliverability nulls** and refer to as such hereafter. Figure 5 sweeps A from 200 to 1400 ms at -24 dBm and shows a pronounced dip at every multiple of 300 ms: the intervals $A \in \{300, 600, 900, 1200\}$ ms average 69.3% deliverability against 92.7% at the surrounding intervals. A cell-clustered permutation test places the gap well beyond chance ($p < 5 \times 10^{-5}$, the floor of a 20,000-shuffle test — no shuffle reproduced the observed gap).

This sweep is a separate run from that of Figure 4, and its design turns the identity $B = T/A$ to advantage. Because fixing any two of the three parameters fixes the third, the run pins B at two levels, 4 and 6 — both below the deliverability ceiling at this power — so that a penalty tracking A must show up at the **same** intervals in both arms, whereas a penalty tracking T would show up at **different** ones. It shows up at the same intervals.

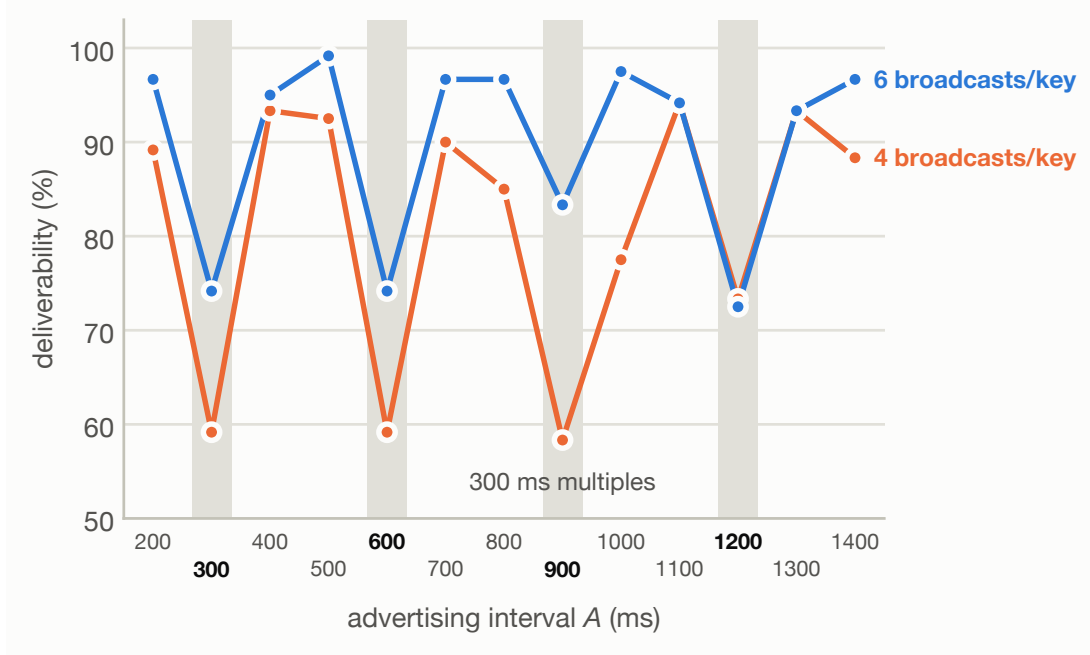


Figure 5: Deliverability against advertising interval A from 200 to 1400 ms at -24 dBm, at 4 and 6 broadcasts per key. Sharp periodic deliverability nulls appear at every multiple of 300 ms.

The explanation is plain arithmetic, and it rests on how BLE scanning works. A scanner does not listen continuously: under the core specification [4] it opens a **scan window** once per fixed **scan interval** and is deaf for the rest of each interval, so a broadcast is heard only if the advertising event happens to fall inside an open window. Suppose a large, coordinated fraction of finder devices open their windows on a fixed period near 300 ms. A transmitter whose advertising interval is an integer multiple of that period keeps landing at the same position within the scan cycle on every event; if that position happens to fall in the deaf gap between windows, every broadcast misses in exactly the same way. A transmitter whose interval is **not** a multiple of 300 ms lands at a shifting position from event to event, samples both open and closed windows, and recovers its deliverability.

Figure 6 makes this precise. It plots deliverability against the number of distinct positions in the scan cycle that a given interval causes the advertiser to reach — a count driven by $\gcd(A, 300\text{ms})$ — at a matched four broadcasts per key. An interval stuck at a single position delivers 48.3%; deliverability then climbs monotonically, 76.1% at two positions and 79.4% at three, to 85.0% at six, saturating somewhere between three and six. Deliverability here is governed entirely by where in the scan cycle the broadcasts land, exactly as the arithmetic predicts.

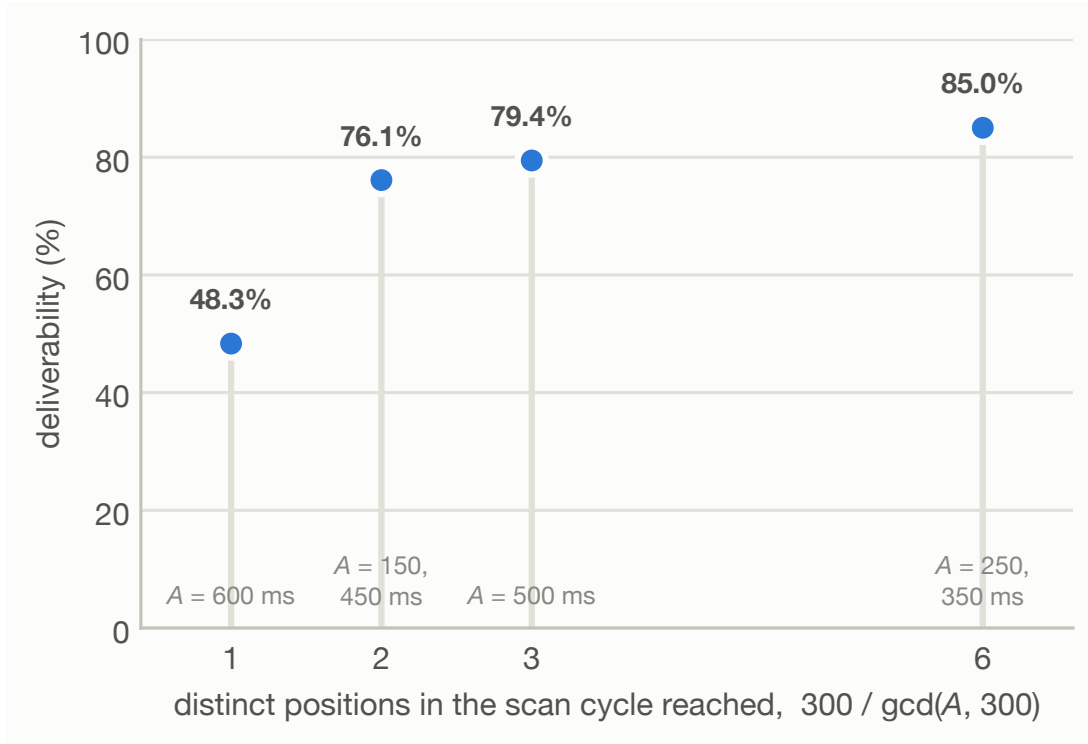


Figure 6: Deliverability against the number of distinct positions in the scan cycle the advertiser reaches, an argument driven by $\gcd(A, 300\text{ms})$.

A natural hope is that an interval stuck at one position could be rescued simply by broadcasting each key more often. It cannot. Figure 7 contrasts a control interval $A = 500$ ms (three positions) with the locked interval $A = 600$ ms (one position) as B rises to 16. The unlocked interval reaches 98.9%, while the locked one stalls at 76.7% — a deficit of over twenty points that survives even at sixteen broadcasts. The reason is that repeated broadcasts at a locked interval are **synchronised repeats**: every one of them lands at the same unfavourable position, so they are not independent opportunities and cannot buy back the loss. The null cannot be escaped with more broadcasts; it can only be escaped by choosing the interval.

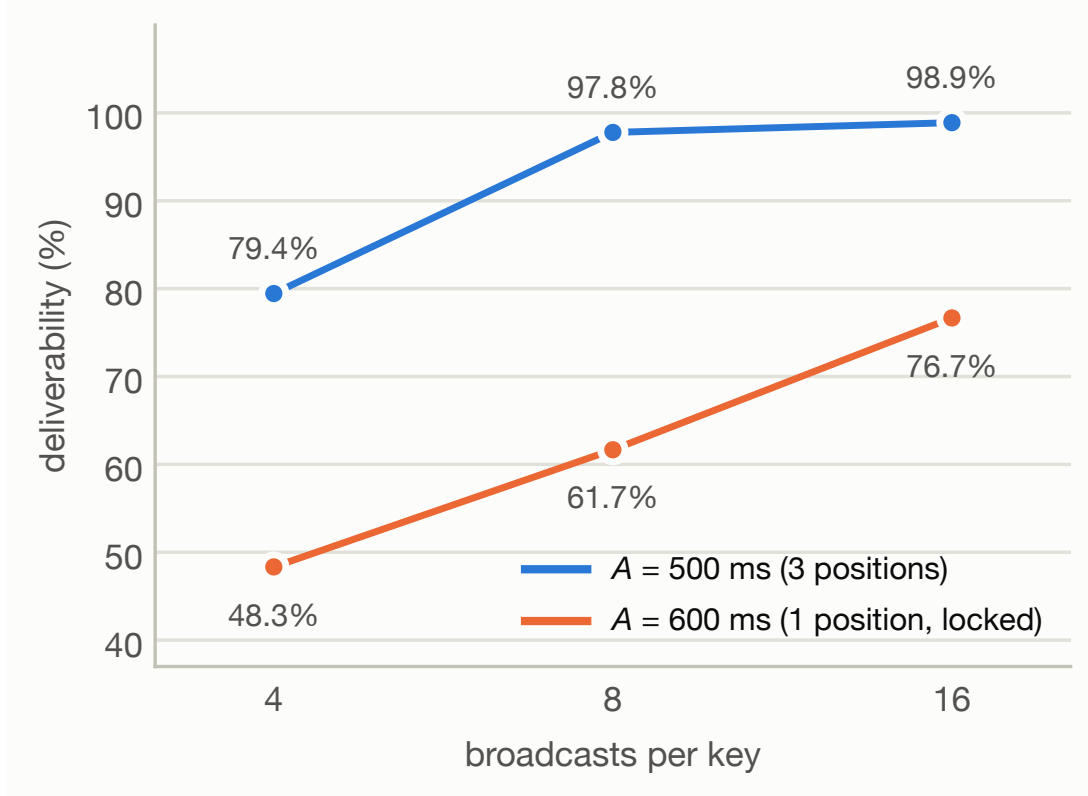


Figure 7: Deliverability as B increases to 16 for a control interval $A = 500$ ms (three positions in the scan cycle) and a locked interval $A = 600$ ms (one position).

5.5. Relay-Bound Latency

Delivery, when it happens, is not prompt. Figure 8 gives the cumulative distribution of propagation latency — from a key’s first broadcast to its first fetchable report — for the factorial run that pins broadcasts per key independently of the rotation period (the advertising interval is scaled with T so that each cell fixes B directly, rather than letting B follow from T/A), at full transmit power. The median is 158 s and the ninetieth percentile 356 s, with a pronounced heavy tail.

Both figures are lower bounds, and deliberately so. Propagation is timed by the live poller, which gives up on a key once its 600 s of patience are spent, and an abandoned key carries no arrival time at all — it drops out of the curve rather than being recorded as slow. The percentiles are therefore computed over the 1522 of 1800 keys the poller happened to time; the 15% it did not time are, by construction, the slow ones, so the true distribution lies to the right of the one plotted. This censoring is a property of the instrument and cannot be undone by re-analysis. It is also why I quote latency from a single run rather than pooling across nights: runs with different poller patience truncate at different points, and their distributions are not comparable.

The delay is not in discovery. Live detections show that a finder device typically hears the advertisement almost immediately — the observation time is close to the send time. The lag is overwhelmingly the time for the finder to upload its encrypted report and for the backend to make it fetchable. The channel is genuinely store-and-forward, and it is the store that dominates.

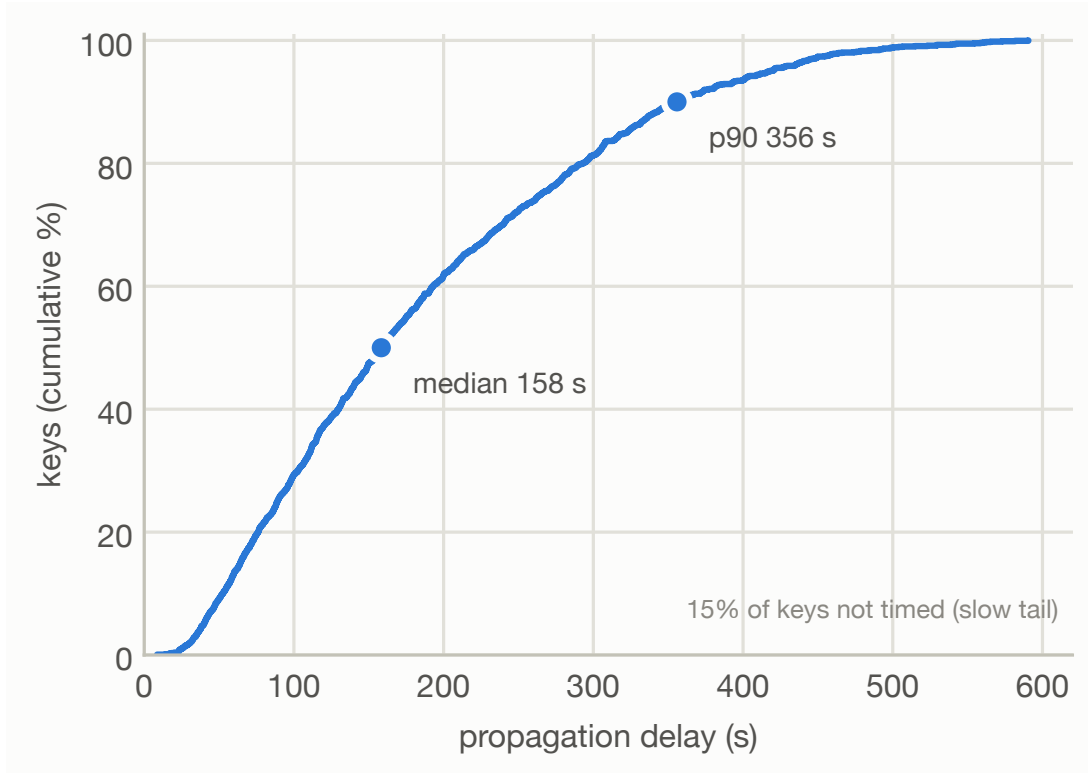


Figure 8: Cumulative distribution of propagation latency, from a key’s broadcast to its first fetchable report, over the factorial run that pins broadcasts per key independently of the rotation period, at full transmit power. The plotted percentiles are lower bounds set by the live poller’s patience.

Latency is also signal-dependent, and this is the one place where the rotation period is not inert. Figure 9 plots median propagation latency against key rotation period and broadcast count within that same run. Twenty broadcasts per key rather than five buys about 40 s off the median (138 s against 178 s), because the earliest of several broadcasts to be relayed sets the clock, and more broadcasts give more early chances. Stronger signal helps too, and here again the contrast must be read within a single run: in the transmit-power run the median was 308 s at +9 dBm against 405 s at −24 dBm. That 97 s gap understates the true effect, because the poller timed 54% of the high-power keys but only 25% of the low-power ones — so the low-power median is the more heavily selected for speed of the two. In every case the distribution stays heavy-tailed: the median moves, but the tail persists.

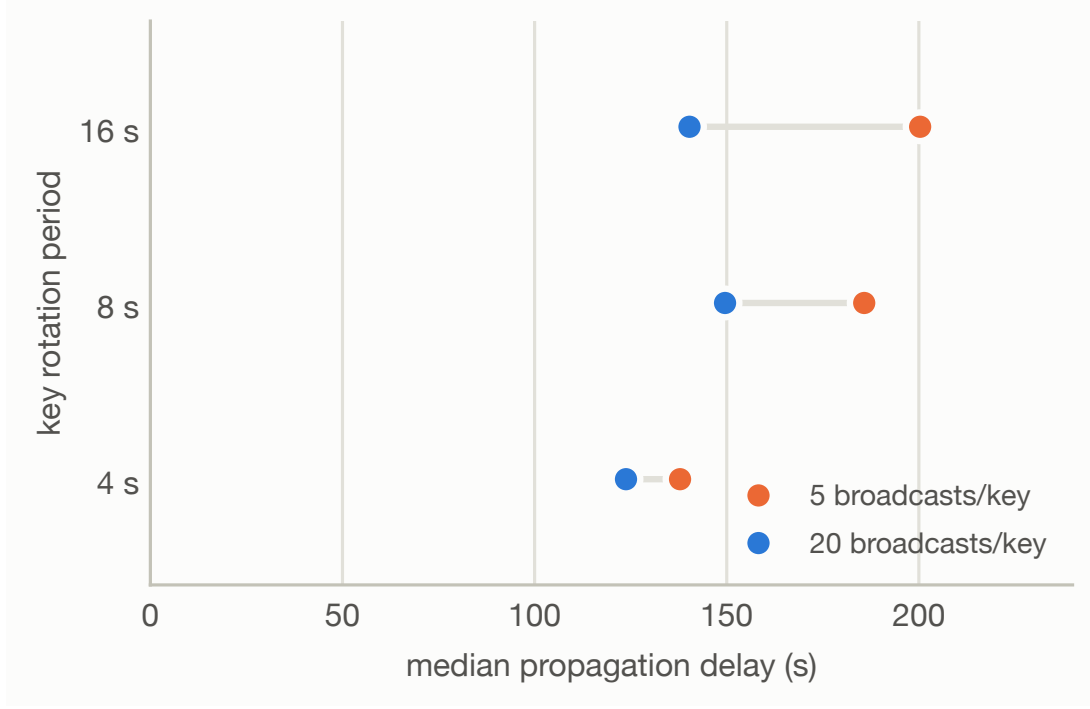


Figure 9: Median propagation latency against key rotation period and broadcast count, from the factorial run that pins broadcasts per key independently of the rotation period.

5.6. The Rate–Reliability Trade-Off

I define goodput as delivered payload bytes per second of wall-clock on-air time. Because each key carries exactly one payload byte, the byte is the natural symbol of the channel, and goodput is simply the delivered-key rate:

$$\text{goodput} = \frac{\text{offline-resweep-delivered keys}}{\text{measured wall-clock on-air seconds}}. \quad (1)$$

Since the delivered count comes from the authoritative offline resweep, goodput inherits the resweep’s reliability rather than the live poller’s. There is no single goodput figure, only a goodput–reliability trade-off. In the full-power, high-reliability regime (rotation periods of 4, 8, and 16 s, at roughly 100% deliverability), goodput ranges from 0.062 to 0.246 byte/s. The fastest point I measured is 1.22 byte/s at 80.6% deliverability ($A = 150$ ms, $T = 600$ ms, -24 dBm), buying rate with reliability; a more defensible operating point, balancing the two, is 0.83 byte/s at 87.8% deliverability ($A = 200$ ms, $T = 1000$ ms, -24 dBm). Full-power sub-second rotation was never actually measured, so a goodput above 1.25 byte/s at high reliability is plausible but remains an extrapolation, not a measurement.

6. Discussion and Interpretation

Taken together, the results describe a specific and coherent kind of channel: a near-lossless, store-and-forward erasure channel with minutes-scale, heavy-tailed latency. It corrupts nothing, loses little at full power, and never delivers quickly. This shape has three direct consequences.

The first is practical — a small set of transmit rules that follow from the deliverability findings.

- **Broadcast each key at least eight times ($B \geq 8$).** This puts deliverability at or near the ceiling at full power, and well up the geometric curve even at low power; it also shaves tens of seconds off the median latency.
- **Never choose an advertising interval at or near a multiple of 300 ms.** Intervals such as 250, 400, 500, and 700 ms sample several positions in the scan cycle and are safe, whereas 300, 600, 900, and 1200 ms are stuck at a single position and cannot be rescued by extra broadcasts.

Together these two rules are close to sufficient for a practitioner to obtain reliable delivery.

The second consequence is inferential. The periodic nulls are, in effect, a measurement of the unobserved scanner population made through the channel itself. Their existence and their 300 ms spacing imply that a substantial, coordinated fraction of finder devices scan on a shared periodic schedule with a period near 300 ms; a population scanning at random offsets could not produce a sharp, interval-locked null. The channel thus reveals a structural property of the crowd that cannot be observed directly.

The third consequence is methodological, and it is the one I would most want a reader to take away. An early live-poller episode showed how an adaptive-timeout observer of a heavy-tailed process can introduce a bias that is not merely noisy but **systematically correlated with the independent variable**, fabricating a clean and plausible trend where none exists. The defence is not a better timeout but a separation of concerns: monitor progress with the adaptive instrument, but settle ground truth with a patience-unbounded resweep.

7. Limitations and Future Work

The study is deliberately deep rather than broad, and its scope bounds its claims. All the measurements come from a single site, a single board, and a single relay population, so the period at which the nulls fall, the absolute deliverability figures, and the latency distribution may all differ elsewhere. Finder density was never manipulated, only observed as a nuisance variable to control against; I can say that it modulates the effects but does not explain them, yet I cannot report a controlled density response. The latency figures are lower bounds set by the poller’s patience and cannot be recovered by re-analysis, because a key the poller stopped waiting for has no recorded arrival time even if it did eventually arrive. The periodic nulls were characterised only at -24 dBm, where the low ceiling makes them visible; their behaviour at full power is unmeasured. And because the anchor cells cannot calibrate a between-night difference, every absolute figure is confined to within-run interpretation.

Two follow-up measurements would be especially valuable. The first is a long-broadcast **escape** experiment at the locked interval $A = 600$ ms, with B pushed as high as 128, to find the broadcast count (if any) at which deliverability finally recovers; since any recovery would come from the scan-cycle position drifting slowly across many events, the recovery point would in turn measure that drift rate. The second is a fine notch-width sweep, in 5–10 ms steps across roughly 590–610 ms, to measure how wide the null actually is, and hence how much margin the “avoid multiples of 300 ms” rule really needs. Beyond measurement, the protocol invites extension: multi-byte framing to carry more than one byte per message identifier, forward error correction to trade goodput for resilience against the residual erasures, and hardening of the shared-secret scheme — rotating the uid for forward secrecy, and adding sender authentication, since today any holder of the uid can both read and forge on the channel.

8. Ethics and Responsible Use

8.1. Network Load

All transmissions in this study were rate-limited to avoid loading the relay network.

8.2. Privacy of Finder-Device Owners

No personal data of finder-device owners was collected: the only owner-side quantity recorded was an aggregate count of nearby finder devices, used as a density covariate. The location reports the protocol handles are encrypted and decryptable only by holders of the pre-shared uid, and at no point does the sender learn anything about the finder devices that relayed its keys beyond the fact that a relay occurred.

8.3. Dual-Use Considerations

The work characterises an existing, public relay mechanism in order to establish whether it can serve as a legitimate low-power uplink for connectivity-starved devices — not to degrade, congest, or abuse it. I acknowledge the dual-use tension that the security literature has documented around exactly this class of mechanism: the same properties that make a relay network a useful uplink can be misused for covert exfiltration or tracking [10], [11], [12]. I regard transparent characterisation, rather than quiet exploitation, as the responsible posture.

8.4. Use of Artificial Intelligence (AI)

AI has been used in various areas of this project, in accordance with the University’s Policy for Use of AI in Teaching and Learning. This subsection describes the aspects and extents to which AI was used.

AI is **never** used in the following areas:

- Ideation of the `sendmy` protocol;
- Secondary research;
- Interpretation of experiment results;
- Prose writing of the final report; and
- This section on the “Use of AI”.

Furthermore, AI is used **sparingly** in the following areas:

- Implementation of the `sendmy` protocol on ESP32 (deciding on what and how to use the IDF-provided API based on my requests); and
- Implementation of the `espbench` project (implementing two-way communication between host and board).

AI is used **generously** in the following areas:

- Implementation of P-224 cryptography on ESP32 (completely refactoring relevant functions in the MicroECC library into a pair of standalone C header/source files);
- Automated execution of testing matrices for `espbench` (monitoring board health, diagnosing network issues, and gracefully resuming testing on failure);
- Raw data analysis (aggregating logs, writing ad-hoc Python scripts to produce relevant figures);

- Design of the poster; and
- Conversion of raw Markdown file into Typst-coded source file.

Regardless of the extents of AI usage, I remain responsible for all project deliverables - the poster and this report - as well as any code written.

9. Conclusion

Prior work showed that Apple’s Find My network can be made to carry arbitrary bytes; this report shows what that channel actually is. It is a pure erasure channel that, at full transmit power and urban finder density, delivers almost everything (about 99.5–99.94% per run) and corrupts nothing (not a single payload error in any run). Its reliability is set chiefly by the number of broadcasts per key, with transmit power fixing the saturation point. Its one sharp failure mode is a family of periodic deliverability nulls at advertising intervals near multiples of 300 ms, explained by the interval lining up with the scanners’ shared listening cycle and inescapable by extra broadcasts. Its cost is latency: delivery is heavy-tailed and relay-bound, with a median of minutes and a tail of many more. The channel is therefore highly reliable but latency-bound — well suited to asynchronous, delay-tolerant messaging from BLE-only devices, and unsuited to anything real-time. The arc of the work is the arc from a proof of concept to a characterised channel, with quantified behaviour and usable design rules.

Bibliography

- [1] F. Bräunlein, ‘Send My: Arbitrary Data Transmission via Apple's Find My Network’. [Online]. Available: <https://positive.security/blog/send-my>[◦]
- [2] A. Heinrich and M. Stute, *OpenHaystack: Build Your Own Find My Network*. (Mar. 2021). [Online]. Available: <https://github.com/seemoo-lab/openhaystack>[◦]
- [3] A. Heinrich, M. Stute, T. Kornhuber, and M. Hollick, ‘Who Can Find My Devices? Security and Privacy of Apple's Crowd-Sourced Bluetooth Location Tracking System’, *Proceedings on Privacy Enhancing Technologies (PoPETs)*, vol. 2021, no. 3, pp. 227–245, Mar. 2021, doi: 10.2478/popets-2021-0045[◦].
- [4] Bluetooth SIG, ‘Bluetooth Core Specification, Version 5.4’, Jan. 2023. [Online]. Available: <https://www.bluetooth.com/specifications/specs/core-specification/>[◦]
- [5] H. Krawczyk and P. Eronen, ‘RFC 5869: HMAC-based Extract-and-Expand Key Derivation Function (HKDF)’, May 2010. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc5869>[◦]
- [6] H. Krawczyk, M. Bellare, and R. Canetti, ‘RFC 2104: HMAC: Keyed-Hashing for Message Authentication’, Feb. 1997. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc2104>[◦]
- [7] L. Chen, D. Moody, A. Regenscheid, A. Robinson, and K. Randall, ‘SP 800-186: Recommendations for Discrete Logarithm-based Cryptography — Elliptic Curve Domain Parameters’, Feb. 2023. doi: 10.6028/NIST.SP.800-186[◦].
- [8] A. Faz-Hernández, S. Scott, N. Sullivan, R. S. Wahby, and C. A. Wood, ‘RFC 9380: Hashing to Elliptic Curves’, Aug. 2023. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc9380>[◦]
- [9] Espressif Systems, ‘ESP-IDF Programming Guide’. Accessed: Aug. 09, 2026. [Online]. Available: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/>[◦]
- [10] A. Gangwal, S. Singh, R. Spolaor, and A. Srivastava, ‘BLEWhisperer: Exploiting BLE Advertisements for Data Exfiltration’, Sep. 26, 2022. doi: 10.1007/978-3-031-17140-6_34[◦].
- [11] H. Alamleh and A. Cantelli-Forti, ‘Defending against BLE-based covert channels in crowdsourced location networks’, *Computers & Security*, vol. 164, May 2026, doi: 10.1016/j.cose.2026.104426[◦].
- [12] Z. Ren, Y. Zhang, B. Liu, and M. Li, ‘Snatcher: Apple Find My Network Exposes Your Lost Devices To Strangers’, in *ACM SIGSAC Conference on Computer and Communications Security*, Jun. 2026. [Online]. Available: <https://arxiv.org/abs/2606.21067>[◦]

Index of Figures

Figure 1	Simplified OF workflow [3]	5
Figure 2	BLE advertising packet structure. ADV Address (2nd last row) and AD Data (last row) fields are what we can freely set.	6
Figure 3	Ambient finder density over the course of a run, sampled continuously as a covariate. Density follows a day-night cycle, with an afternoon and evening trough and an early-morning peak.	11
Figure 4	Deliverability against broadcasts per key $B \in \{1, 2, 4, 8, 16\}$ at -24 dBm and $+9$ dBm. Misses decay approximately geometrically in B	14
Figure 5	Deliverability against advertising interval A from 200 to 1400 ms at -24 dBm, at 4 and 6 broadcasts per key. Sharp periodic deliverability nulls appear at every multiple of 300 ms.	15
Figure 6	Deliverability against the number of distinct positions in the scan cycle the advertiser reaches, an argument driven by $\gcd(A, 300\text{ms})$	16
Figure 7	Deliverability as B increases to 16 for a control interval $A = 500$ ms (three positions in the scan cycle) and a locked interval $A = 600$ ms (one position).	17
Figure 8	Cumulative distribution of propagation latency, from a key's broadcast to its first fetchable report, over the factorial run that pins broadcasts per key independently of the rotation period, at full transmit power. The plotted percentiles are lower bounds set by the live poller's patience.	18
Figure 9	Median propagation latency against key rotation period and broadcast count, from the factorial run that pins broadcasts per key independently of the rotation period.	19

Index of Tables

Table 1	Notation and terminology used throughout. A transmitter is characterised by the triple (A, T, B) together with its transmit power.	4
Table 2	Find My BLE advertising payload format. The first 6 bytes of the key are used as the BLE device address.	6